Universidad de San Carlos de Guatemala
Escuela de Ingeniería en Ciencias y Sistemas, Facultad de Ingeniería
Introducción a la programación y computación 2, 2er. Semestre 2026

SISTEMA DE AUTORIZACIÓN Y CONTROL DE DOCUMENTOS TRIBUTARIOS ELECTRÓNICOS MEDIANTE UNA API WEB

202112134 – Lorena Abigail Arrecis Martinez

Resumen

El presente ensayo describe el desarrollo de un sistema de autorización y control de Documentos Tributarios Electrónicos, realizado como Proyecto 3 del curso de Introducción a la Programación y Computación 2. La solución fue construida utilizando C#, ASP.NET Core, Razor Pages, archivos JSON y comunicación mediante una API HTTP. El sistema permite administrar contribuyentes, validar números de identificación tributaria, recibir solicitudes de autorización, calcular y verificar el IVA y el total de una factura, generar códigos de autorización y almacenar información para consultas posteriores. Además, incorpora reportes por fechas, resúmenes de IVA, consultas de documentos emitidos y recibidos, gráficas y generación de reportes en formato PDF. Durante el desarrollo se aplicaron conceptos de programación orientada a objetos, separación de responsabilidades, persistencia de datos y validación de información. El proyecto permitió comprender cómo diferentes componentes pueden comunicarse para construir una aplicación distribuida, manteniendo organizada la lógica del sistema y facilitando su mantenimiento.

Palabras clave: API, C#, facturación, ASP.NET, programación

Abstract

This essay describes the development of an authorization and control system for Electronic Tax Documents, created as Project 3 for the Introduction to Programming and Computing 2 course. The solution was built using C#, ASP.NET Core, Razor Pages, JSON files, and communication through an HTTP API. The system manages taxpayers, validates tax identification numbers, receives authorization requests, calculates and verifies VAT and invoice totals, generates authorization codes, and stores information for later queries. It also includes date-based reports, VAT summaries, queries for issued and received documents, charts, and PDF report generation. During development, concepts such as object-oriented programming, separation of responsibilities, data persistence, and information validation were applied. The project made it possible to understand how different software components can communicate to build a distributed application while keeping the system logic organized and making maintenance easier.

Keywords: API, C#, invoicing, ASP.NET, programming

Introducción

El procesamiento de documentos tributarios requiere validar información antes de aceptar una transacción. Datos como el NIT del emisor y receptor, el valor de una factura, el IVA, el total y la referencia del documento deben cumplir reglas específicas para evitar información incorrecta o duplicada. En este proyecto se desarrolló un sistema que simula parte de este proceso mediante una API construida con ASP.NET Core y una interfaz web implementada con Razor Pages. El objetivo principal fue aplicar los conocimientos adquiridos sobre programación orientada a objetos, estructuras de datos, persistencia y comunicación entre sistemas. A través de esta solución se implementaron operaciones para registrar contribuyentes, autorizar documentos, realizar consultas y generar reportes. El desarrollo permitió observar la importancia de organizar correctamente las responsabilidades de cada clase y validar los datos antes de almacenarlos..

Desarrollo del tema

1. PLANTEAMIENTO Y FUNCIONAMIENTO DEL SISTEMA

El sistema desarrollado tiene como objetivo simular el proceso de recepción y autorización de documentos tributarios. Para conseguirlo se dividió la aplicación en dos componentes principales: un backend encargado de procesar la información y un frontend utilizado por el usuario para interactuar con el sistema.

El backend fue desarrollado utilizando ASP.NET Core. Su función principal es recibir solicitudes HTTP, aplicar las reglas del sistema y retornar una respuesta al usuario. Por otra parte, el frontend fue desarrollado mediante Razor Pages, permitiendo disponer de páginas web desde las cuales se pueden cargar archivos, realizar consultas, generar reportes e inicializar la información almacenada.

Esta separación permite que la lógica del negocio no dependa directamente de la interfaz gráfica. El frontend únicamente solicita operaciones al backend y presenta el resultado obtenido. De esta forma, la API podría ser utilizada posteriormente por otra aplicación sin necesidad de modificar la lógica principal.

La información se transmite principalmente utilizando JSON. Este formato facilita representar objetos y listas mediante texto estructurado. Por ejemplo, para registrar contribuyentes se envía una lista que contiene el NIT y el nombre de cada persona o empresa. El servidor recibe dicha información, la convierte en objetos de C# y posteriormente realiza las validaciones correspondientes.

2. REGISTRO TRIBUTARIO UNIFICADO Y VALIDACIÓN DEL NIT

Uno de los primeros módulos implementados fue el Registro Tributario Unificado, denominado RTU dentro del sistema. Este módulo permite registrar los contribuyentes que posteriormente podrán participar como emisores o receptores de documentos tributarios.

Cada contribuyente posee principalmente un número de identificación tributaria y un nombre. Antes de registrar un contribuyente, el programa verifica que el NIT sea válido. Esta validación no consiste únicamente en comprobar que el campo tenga información, sino que utiliza el procedimiento matemático correspondiente para validar su dígito verificador.

Para realizar esta operación se creó un servicio específico denominado NitService. Separar esta responsabilidad permite reutilizar la validación en distintos procesos. Por ejemplo, el mismo servicio es utilizado durante la carga del RTU y durante la autorización de documentos.

Cuando se recibe un archivo de contribuyentes pueden ocurrir tres situaciones. Si el NIT es válido y todavía no existe, el contribuyente se registra como nuevo. Si ya existe, su información puede ser actualizada. Finalmente, cuando el NIT no cumple las condiciones de validación, no se modifica la información almacenada y se contabiliza como inválido.

Al finalizar el proceso, el sistema genera un resumen indicando la cantidad de contribuyentes nuevos, actualizados y los NIT inválidos encontrados. Este resultado permite que el usuario conozca inmediatamente el resultado de la operación.

3. AUTORIZACIÓN DE DOCUMENTOS TRIBUTARIOS

La autorización de documentos representa una de las partes principales del proyecto. Para realizar esta operación, el sistema recibe información como la fecha y hora, la referencia del documento, NIT del emisor, NIT del receptor, valor de la factura, IVA y total.

Antes de generar una autorización se deben cumplir diferentes condiciones. En primer lugar, la referencia debe existir, tener una longitud permitida y no haber sido utilizada anteriormente. En este proyecto la referencia se considera única para todo el sistema, independientemente del contribuyente que envíe la solicitud.

Posteriormente se valida el NIT del emisor y del receptor. Además de tener un formato correcto, ambos contribuyentes deben encontrarse registrados previamente en el RTU. Esto evita autorizar documentos asociados a contribuyentes inexistentes.

El valor de la factura debe ser positivo. Con base en dicho valor se calcula el Impuesto al Valor Agregado utilizando una tasa del doce por ciento. El resultado se redondea a dos posiciones decimales y se compara con el IVA recibido. Finalmente, el total debe coincidir con la suma del valor más el IVA.

La operación puede expresarse de manera sencilla mediante:

IVA = VALOR × 0.12

TOTAL = VALOR + IVA

Si alguna de estas reglas no se cumple, el documento se almacena como rechazado junto con los errores detectados. Registrar también las solicitudes rechazadas resulta importante porque posteriormente permite generar estadísticas sobre los diferentes tipos de errores encontrados.

Cuando todas las validaciones son correctas se genera un código de autorización. Este código está formado por la fecha del documento y un número correlativo diario. El correlativo comienza desde uno para cada fecha y aumenta con cada documento aprobado.

Por ejemplo, un código generado puede tener la siguiente forma:

2026102100000001

Los primeros ocho caracteres representan la fecha en formato año, mes y día, mientras que los últimos ocho corresponden al correlativo.

También se consideró el caso donde varias solicitudes puedan llegar prácticamente al mismo tiempo. Para evitar generar correlativos duplicados, el acceso al proceso de escritura y generación del código se controla antes de guardar los registros.

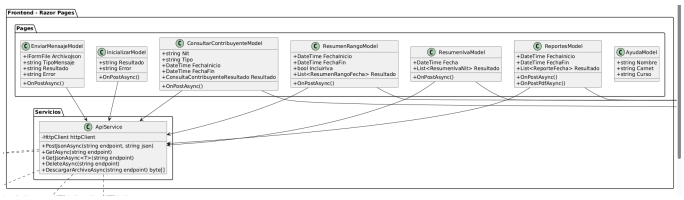


Figura 1. Clases principales del frontend y comunicación mediante ApiService.

Fuente: elaboración propia

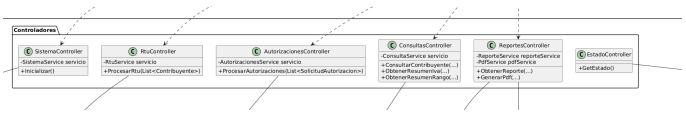


Figura 2. Controladores principales de la API del sistema.

Fuente: elaboración propia

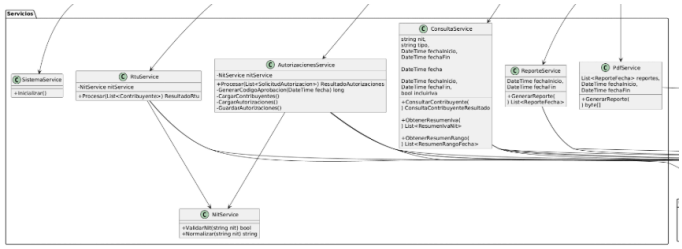


Figura 3. Servicios encargados de la lógica de negocio del sistema.

Fuente: elaboración propia

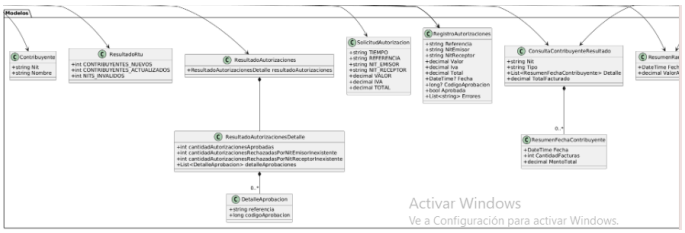


Figura 4. Modelos utilizados para representar la información del sistema

Fuente: elaboración propia

Conclusiones

El desarrollo del sistema permitió aplicar de manera práctica los principales conceptos de programación orientada a objetos estudiados durante el curso. La separación entre modelos, servicios y controladores permitió mantener organizada la lógica y facilitó la integración entre los diferentes componentes.

La implementación de una API HTTP permitió comprender cómo dos aplicaciones independientes pueden intercambiar información utilizando solicitudes y respuestas. Esta estructura permite que el backend pueda ser utilizado no solamente por el frontend desarrollado con Razor Pages, sino también por otras aplicaciones capaces de consumir servicios HTTP.

Referencias bibliográficas

Microsoft. (2026). ASP.NET Core documentation. Microsoft Learn.

QuestPDF. (2026). QuestPDF documentation.

.